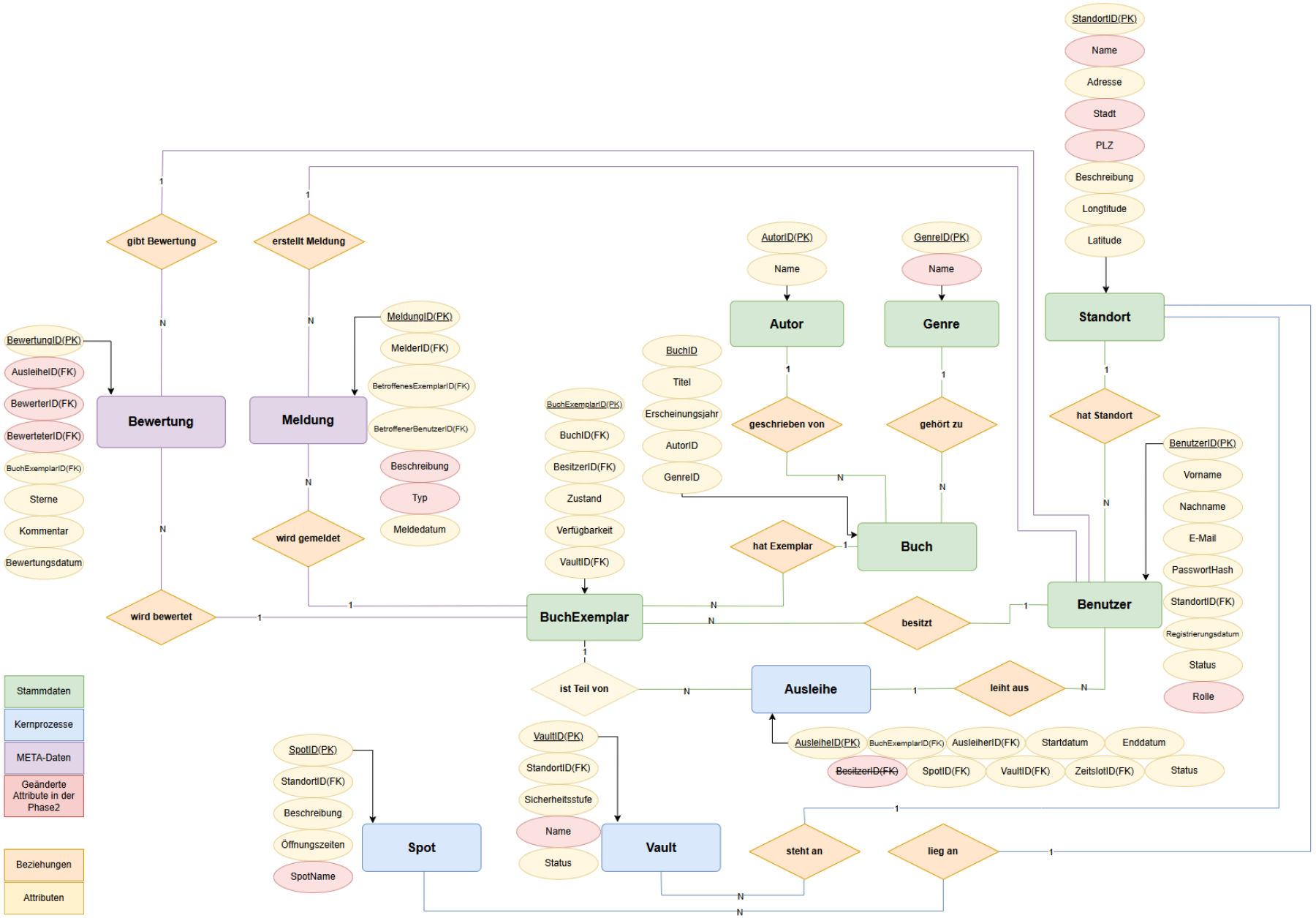
IMPLEMENTIERUNG EINES RELATIONALEN DATENBANKSYSTEMS FÜR EIN BUCH-TAUSCH-APP

VLADYSLAV BONDAR – 30.03.2026 – DLBDSPBDM01_D

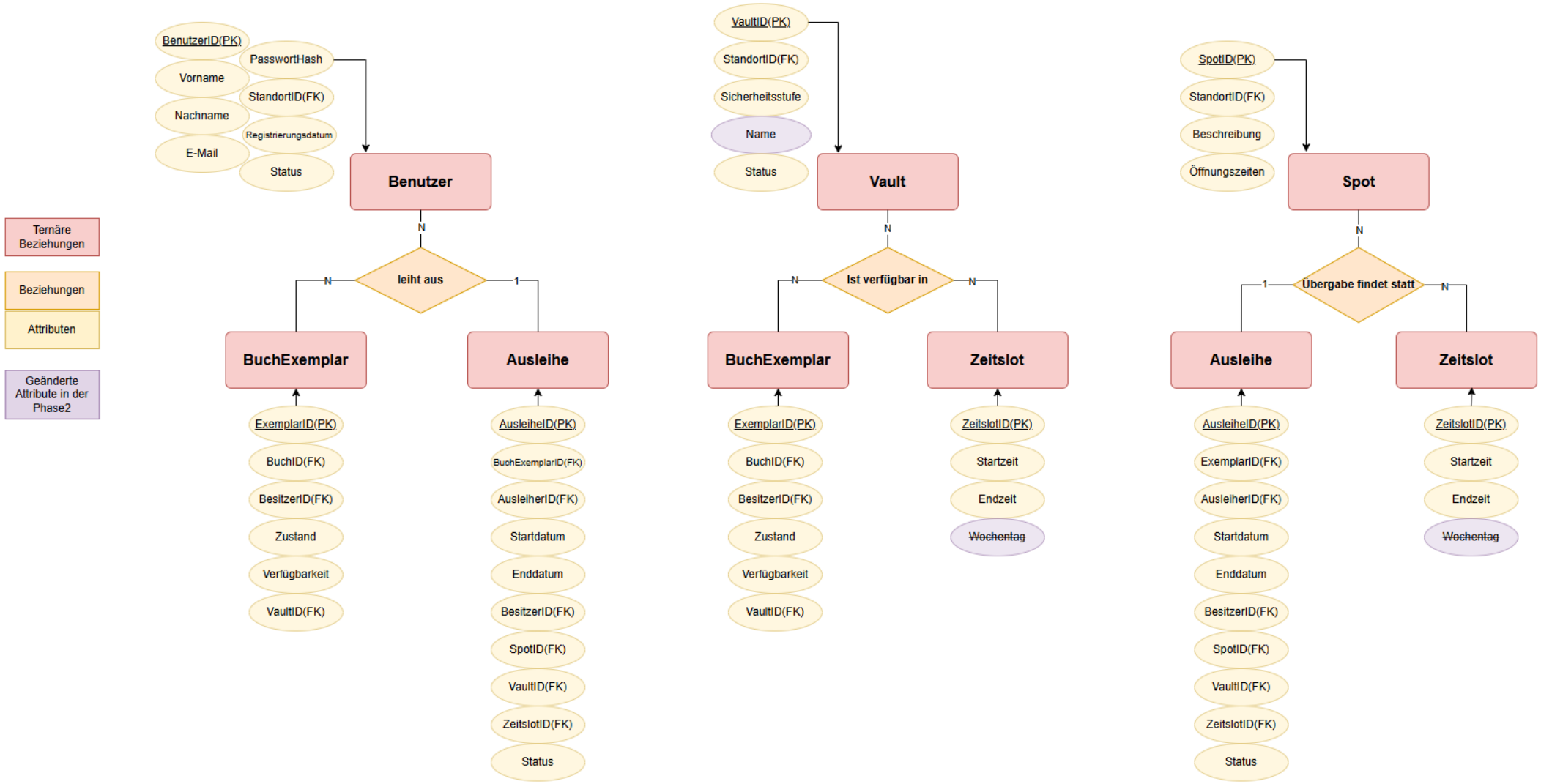
INHALT:

- ER-Modell
- SQL-Implementierung – Tabellen erstellen & befüllen
- SQL-Implementierung – Testfälle
- Komplexer Testfall
- Zusammenfassung

ER-Modell – Binäre Beziehungen



ER-Modell – Ternäre Beziehungen



ER-Modell – Änderungen gegenüber Phase 1

Im Vergleich zur Modellierung aus Phase 1 wurden in Phase 2 mehrere Anpassungen vorgenommen, die sowohl der fachlichen Präzisierung als auch den Anforderungen des Tutors dienen. Die Änderungen sind vollständig begründet und verletzen nicht die ursprüngliche Modelllogik.

Tabelle: Standort

Die Attribute *Beschreibung* und *Longitude* (ggf. Latitude) wurden aus Phase 1 übernommen. Ergänzend wurden *Stadt, PLZ* und *Adresse* hinzugefügt, um eine genauere geographische Zuordnung und verbesserte Filtermöglichkeiten zu ermöglichen. Diese Erweiterung erhöht die Nutzbarkeit des Modells, ohne die ursprüngliche Struktur zu verändern.

Tabelle: Benutzer

Das Attribut *Rolle* wurde ergänzt, da in Phase 1 bereits zwischen normalen Benutzern und Administratoren unterschieden wurde. Die Implementierung macht diese Rollenlogik explizit sichtbar.

Tabelle: Genre

In Phase 1 war das Genre über eine allgemeine Beschreibung modelliert. In Phase 2 wurde das Attribut *Name* ergänzt, da Genres üblicherweise über ihre_Bezeichnung identifiziert werden. Das bisherige Attribut *Beschreibung* entfällt dadurch.

Tabelle: Spot

Das Attribut *SpotName* wurde ergänzt, um Übergabeorte eindeutig benennen und klar voneinander unterscheiden zu können.

Tabelle: Vault

Das Attribut *Name* wurde ergänzt, um die strategische Benennung der Vaults (z. B. „RED-1“) abzubilden. Das Phase-1-Attribut *Status* wurde wieder aufgenommen, da es den Betriebszustand eines Vaults beschreibt.

Tabelle: Ausleihe

Das Attribut *BesitzerID* wurde aus der Tabelle *Ausleihe* entfernt, da es vollständig redundant war. Der Besitzer eines Exemplars ist bereits eindeutig in der Tabelle *BuchExemplar* hinterlegt.

Tabelle: Bewertung

Das Attribut *BuchExemplarID* wurde ergänzt, da Bewertungen in Phase 1 sowohl einem Ausleihvorgang als auch einem konkreten Exemplar zugeordnet waren. Zusätzlich wurde das Attribut *Bewertungsdatum* aufgenommen.

Tabelle: Meldung

Die Attribute *Typ* und *Beschreibung* wurden beibehalten, da sie die Art und den Inhalt einer Meldung klar beschreiben. Die Struktur entspricht vollständig der Phase-1-Modellierung. Fälschliche Attribute wie *Art*, *Details* oder *AusleiheID* wurden bewusst nicht übernommen, da sie nicht zur Meldung gehören.

ER-Modell - Erklärung

Das ER-Modell bildet alle zentralen Prozesse des Buch-Tausch-Apps strukturiert und normalisiert ab. Die klare Trennung zwischen *Buch* und *BuchExemplar* stellt sicher, dass logische Werke und physische Kopien nicht vermischt werden. Dadurch können mehrere Benutzer dasselbe Buch besitzen, während jedes Exemplar eindeutig einem Besitzer zugeordnet bleibt.

Die Entität *Ausleihe* verbindet Ausleiher, Exemplar, Übergabeort und Zeitslot. Übergaben können entweder an einem öffentlichen *Spot* oder in einem sicheren *Vault* stattfinden, was durch eine XOR-Beziehung technisch erzwungen wird.

Bewertungen beziehen sich immer auf eine konkrete Ausleihe und enthalten sowohl Bewerter als auch Bewerteten. Meldungen erfassen Probleme zu Exemplaren und betroffenen Benutzern und bilden damit reale Vorfälle präzise ab. Insgesamt ist das Modell vollständig normalisiert, vermeidet Redundanz und bildet alle fachlichen Rollen und Abläufe konsistent ab.

Inhalt:

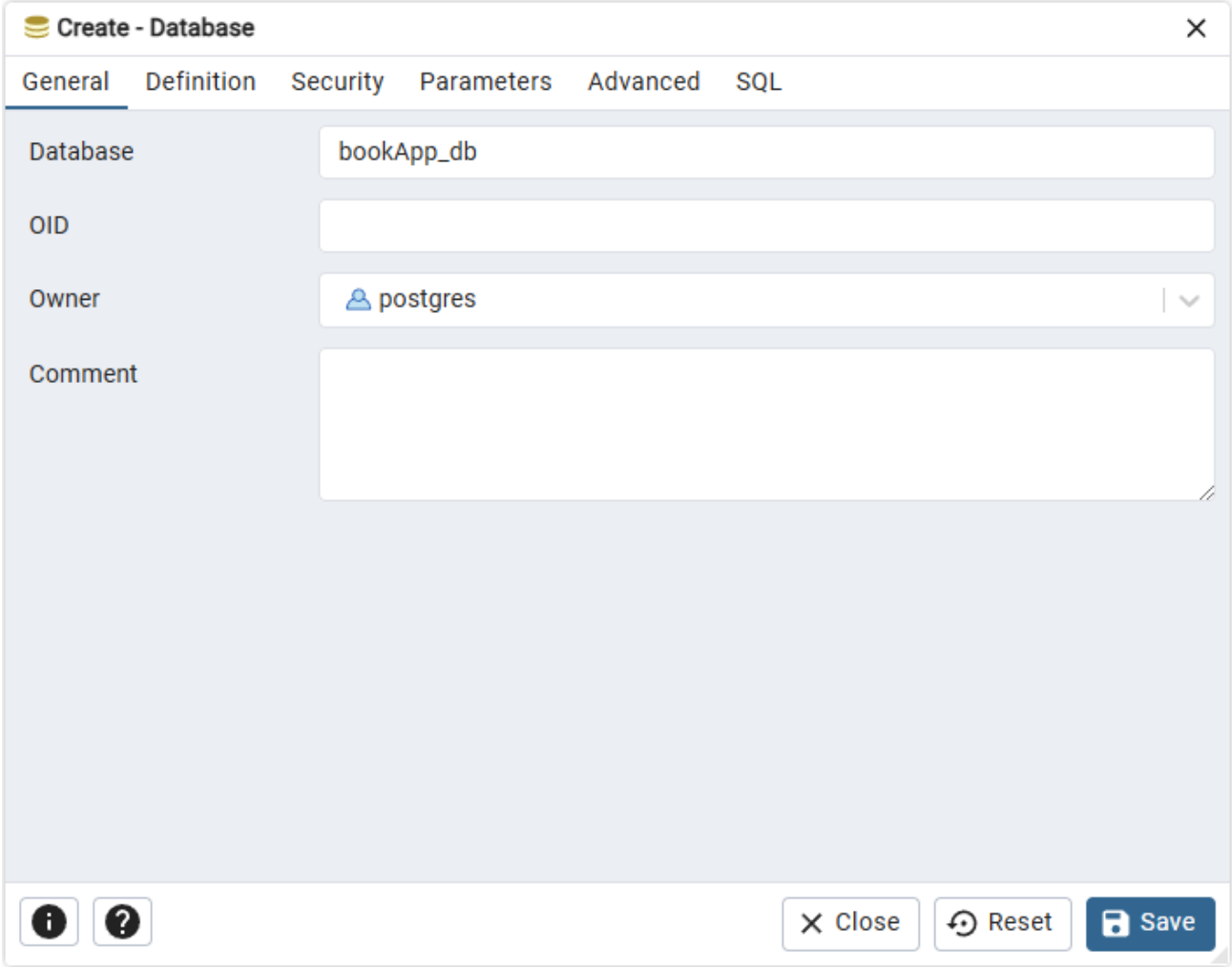
1. Standort
2. Benutzer
3. Autor
4. Genre
5. Buch
6. Spot
7. Vault
8. Zeitslot
9. BuchExemplar
10. Ausleihe
11. Bewertung
12. Meldung

SQL-Implementierung – Strategie

Wichtige Entscheidungen:

- Verwendung von „SERIAL“ für Primärschlüssel
- Fremdschlüssel mit „REFERENCES“
- UNIQUE-Constraints (Email, Genre.Name)
- CHECK-Contraints für Spot/Vault-XOR
- Normalisierung: BesitzerID aus Ausleihe entfernt
- Bewertung referenziert *Ausleihe* und *BuchExemplar*

Die Datenbank bookApp_db wurde im PostgreSQL-DBMS eingerichtet. Im nächsten Schritt werden die entsprechenden Tabellen aus der Phase 1 inkl. Anpassungen in der Datenbank erstellt.



SQL-Implementierung – Tabelle Standort

Tabelle erstellen:

Query | Query History

```sql
-- Ein physischer Ort innerhalb der Stadt (z. B. Stadtteil, Bibliothek, Platz), der als Standort für einen Vault oder Spot verwendet werden kann.

CREATE TABLE Standort (
    StandortID    SERIAL PRIMARY KEY,
    Name          VARCHAR(100) NOT NULL,
    Adresse       VARCHAR(200) NOT NULL, -- Das Attribut wird nicht weiter detailliert, da es voraussichtlich nicht redundant auftreten wird.
    Stadt         VARCHAR(100) NOT NULL,
    PLZ           VARCHAR(10)  NOT NULL,
    Beschreibung  VARCHAR(200),
    Longitude     DECIMAL(9,6),
    Latitude      DECIMAL(9,6)
);

```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Standort (Name, Adresse, Stadt, PLZ, Beschreibung, Longitude, Latitude) VALUES
('Hauptbibliothek Stuttgart', 'Schlossplatz 1', 'Stuttgart', '70173', 'Zentraler Standort', 9.1770, 48.7761),
('Campus Bibliothek Karlsruhe', 'Universitätsstraße 32', 'Karlsruhe', '76131', 'Bibliothek am Campus', 8.4044, 49.0134),
('Bücherhalle Mannheim', 'Marktplatz 5', 'Mannheim', '68159', 'Stadtbibliothek', 8.4660, 49.4875),
('Lesecenter Freiburg', 'Rathausgasse 12', 'Freiburg', '79098', 'Altstadt Standort', 7.8522, 47.9961),
('Mediathek Ulm', 'Hafenbad 10', 'Ulm', '89073', 'Zentralbibliothek', 9.9937, 48.3984),
('Bücherpunkt Tübingen', 'Neckargasse 7', 'Tübingen', '72070', 'Innenstadt', 9.0536, 48.5227),
('Lesehaus Heilbronn', 'Kaiserstraße 20', 'Heilbronn', '74072', 'Stadtmitte', 9.2180, 49.1427),
('Bibliothek Pforzheim', 'Westliche 45', 'Pforzheim', '75172', 'Zentraler Standort', 8.6989, 48.8922),
('Leseraum Ludwigsburg', 'Wilhelmstraße 3', 'Ludwigsburg', '71638', 'Altstadt', 9.1885, 48.8950),
('Bücherinsel Esslingen', 'Bahnhofstraße 8', 'Esslingen', '73728', 'Bahnhofsnah', 9.3074, 48.7396);
```

SQL-Implementierung – Tabelle Standort - Testfall

Testfall:

Query | Query History

```
1  SELECT * FROM Standort LIMIT 5;
```

Ergebnis:

Data Output | Messages | Notifications

	standortid [PK] integer	name character varying (100)	adresse character varying (200)	stadt character varying (100)	plz character varying (10)	beschreibung character varying (200)	longitude numeric (9,6)	latitude numeric (9,6)
1	1	Hauptbibliothek Stuttgart	Schlossplatz 1	Stuttgart	70173	Zentraler Standort	9.177000	48.776100
2	2	Campus Bibliothek Karlsru...	Universitätsstraße 32	Karlsruhe	76131	Bibliothek am Campus	8.404400	49.013400
3	3	Bücherhalle Mannheim	Marktplatz 5	Mannheim	68159	Stadtbibliothek	8.466000	49.487500
4	4	Lesecenter Freiburg	Rathausgasse 12	Freiburg	79098	Altstadt Standort	7.852200	47.996100
5	5	Mediathek Ulm	Hafenbad 10	Ulm	89073	Zentralbibliothek	9.993700	48.398400

SQL-Implementierung – Tabelle Benutzer

Tabelle erstellen:

Query | Query History

```sql
-- Die im System registrierten Benutzer

CREATE TABLE Benutzer (
    BenutzerID          SERIAL PRIMARY KEY,
    Vorname             VARCHAR(50) NOT NULL,
    Nachname            VARCHAR(50) NOT NULL,
    Email               VARCHAR(150) UNIQUE NOT NULL,
    PasswortHash        VARCHAR(200) NOT NULL,
    Registrierungsdatum DATE NOT NULL,
    Status              VARCHAR(20) NOT NULL, -- z.B. aktiv, gesperrt, deaktiviert, gelöscht, etc.
    Rolle               VARCHAR(50) NOT NULL, -- Administrator oder Benutzer
    StandortID          INT NOT NULL REFERENCES Standort(StandortID) --
);
-- Der Standort eines Benutzers wird als Fremdschlüssel gespeichert, weil mehrere Benutzer denselben Standort nutzen können.
-- Dadurch vermeidet man doppelte Einträge, und die Standortdaten müssen nur einmal gepflegt werden.
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Benutzer (Vorname, Nachname, Email, PasswortHash, Registrierungsdatum, Status, Rolle, StandortID) VALUES
('Anton', 'Muster', 'anton@beispielmail.de', 'BeispielKennwort', '2024-01-10', 'aktiv', 'Benutzer', 1),
('Berta', 'Muster', 'berta@beispielmail.de', 'BeispielKennwort', '2024-01-12', 'aktiv', 'Benutzer', 2),
('Charlie', 'Muster', 'charlie@beispielmail.de', 'BeispielKennwort', '2024-01-15', 'aktiv', 'Benutzer', 3),
('Dora', 'Muster', 'dora@beispielmail.de', 'BeispielKennwort', '2024-01-20', 'aktiv', 'Benutzer', 4),
('Emil', 'Muster', 'emil@beispielmail.de', 'BeispielKennwort', '2024-01-25', 'aktiv', 'Benutzer', 5),
('Frieda', 'Muster', 'frieda@beispielmail.de', 'BeispielKennwort', '2024-02-01', 'aktiv', 'Benutzer', 6),
('Gustav', 'Muster', 'gustav@beispielmail.de', 'BeispielKennwort', '2024-02-05', 'aktiv', 'Benutzer', 7),
('Hanna', 'Muster', 'hanna@beispielmail.de', 'BeispielKennwort', '2024-02-10', 'aktiv', 'Benutzer', 8),
('Ida', 'Muster', 'ida@beispielmail.de', 'BeispielKennwort', '2024-02-15', 'aktiv', 'Benutzer', 9),
('Julius', 'Muster', 'julius@beispielmail.de', 'BeispielKennwort', '2024-02-20', 'aktiv', 'Administrator', 10);
```

SQL-Implementierung – Tabelle Benutzer - Testfall

Testfall:

Query | Query History

```
1  SELECT Vorname, Nachname, Rolle FROM Benutzer;
```

Ergebnis:

Data Output | Messages | Notifications

	vorname character varying (50)	nachname character varying (50)	rolle character varying (50)
1	Anton	Muster	Benutzer
2	Berta	Muster	Benutzer
3	Charlie	Muster	Benutzer
4	Dora	Muster	Benutzer
5	Emil	Muster	Benutzer
6	Frieda	Muster	Benutzer
7	Gustav	Muster	Benutzer
8	Hanna	Muster	Benutzer
9	Ida	Muster	Benutzer
10	Julius	Muster	Administrator

SQL-Implementierung – Tabelle Autor

Tabelle erstellen:

Query | Query History

```sql
-- Buchautoren
CREATE TABLE Autor (
    AutorID SERIAL PRIMARY KEY,
    Name    VARCHAR(150) NOT NULL
);
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Autor (Name) VALUES
('J. K. Rowling'),
('George Orwell'),
('J. R. R. Tolkien'),
('Stephen King'),
('Agatha Christie'),
('Dan Brown'),
('Suzanne Collins'),
('Brandon Sanderson'),
('Haruki Murakami'),
('Isaac Asimov');
```

SQL-Implementierung – Tabelle Autor - Testfall

Testfall:

Query | Query History

```
1  SELECT * FROM AUTOR LIMIT 3;
```

Ergebnis:

Data Output | Messages | Notifications

	autorid [PK] integer	name character varying (150)
1	1	J. K. Rowling
2	2	George Orwell
3	3	J. R. R. Tolkien

SQL-Implementierung – Tabelle Genre

Tabelle erstellen:

Query | Query History

```sql
-- Das Genre wird als eigene Entität modelliert, um redundante Daten zu vermeiden.
CREATE TABLE Genre (
    GenreID      SERIAL PRIMARY KEY,
    Name         VARCHAR(100) NOT NULL UNIQUE,
    Beschreibung VARCHAR(200)
);
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Genre (Name, Beschreibung) VALUES
('Fantasy', 'Magische Welten'),
('Dystopie', 'Zukunftsvisionen'),
('Science Fiction', 'Technologische Zukunft'),
('Krimi', 'Detektivgeschichten'),
('Thriller', 'Spannung und Action'),
('Romantik', 'Liebesgeschichten'),
('Abenteuer', 'Reise und Abenteuer'),
('Horror', 'Übernatürliches Grauen'),
('Drama', 'Emotionale Geschichten'),
('Historisch', 'Geschichten aus der Vergangenheit');
```

SQL-Implementierung – Tabelle Genre - Testfall

Testfall:

Query | Query History

```
1  SELECT * FROM Genre LIMIT 3;
```

Ergebnis:

Data Output | Messages | Notifications

	genreid [PK] integer	name character varying (100)	beschreibung character varying (200)
1	1	Fantasy	Magische Welten
2	2	Dystopie	Zukunftsvisionen
3	3	Science Fiction	Technologische Zukunft

SQL-Implementierung – Tabelle Buch

Tabelle erstellen:

Query | Query History

```sql
CREATE TABLE Buch ( -- Das inhaltliche Buchwerk, nicht die physische Ausgabe.
    BuchID SERIAL PRIMARY KEY,
    Titel VARCHAR(200) NOT NULL,
    Erscheinungsjahr INT NOT NULL,
    AutorID INT NOT NULL REFERENCES Autor(AutorID),
    GenreID INT NOT NULL REFERENCES Genre(GenreID)
    -- Autor und Genre werden ausgelagert, weil sie bei vielen Büchern identisch sind.
    -- Über Fremdschlüssel lassen sich Änderungen zentral durchführen, ohne dass man Daten mehrfach anpassen muss.
);
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Buch (Titel, Erscheinungsjahr, AutorID, GenreID) VALUES
('Harry Potter 1', 1997, 1, 1),
('1984', 1949, 2, 2),
('Der Hobbit', 1937, 3, 7),
('Es', 1986, 4, 8),
('Mord im Orientexpress', 1934, 5, 4),
('Sakrileg', 2003, 6, 5),
('Die Tribute von Panem', 2008, 7, 2),
('Mistborn', 2006, 8, 1),
('Kafka am Strand', 2002, 9, 9),
('Foundation', 1951, 10, 3);
```

SQL-Implementierung – Tabelle Buch - Testfall

Testfall:

Query | Query History

```
1  SELECT Titel, Erscheinungsjahr FROM Buch LIMIT 5;
```

Ergebnis:

Data Output | Messages | Notifications

	titel character varying (200)	erscheinungsjahr integer
1	Harry Potter 1	1997
2	1984	1949
3	Der Hobbit	1937
4	Es	1986
5	Mord im Orientexpress	1934

SQL-Implementierung – Tabelle Spot

Tabelle erstellen:

Query | Query History

```sql
CREATE TABLE Spot ( -- Ein sicherer Übergabeort an einem öffentlichen Platz, beispielsweise eine Parkbank, ein Café oder ein Eingang zur Bahn.
    SpotID SERIAL PRIMARY KEY,
    StandortID INT NOT NULL REFERENCES Standort(StandortID),
    SpotName VARCHAR(100) NOT NULL, -- -- z.B. "Parkbank Nord", "Eingang Stadtbibliothek"
    Beschreibung VARCHAR(200),
    Öffnungszeiten VARCHAR(100)
);
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Spot (StandortID, SpotName, Beschreibung, Öffnungszeiten) VALUES
(1, 'Schlossplatz', 'Zentraler Übergabeort', '08:00-20:00'),
(2, 'Campus Süd', 'Vor dem Haupteingang', '07:00-22:00'),
(3, 'Marktplatz', 'Öffentlicher Platz', '09:00-18:00'),
(4, 'Rathausplatz', 'Altstadtbereich', '09:00-19:00'),
(5, 'Hafenbad', 'Innenstadt', '08:00-18:00'),
(6, 'Neckargasse', 'Zentraler Treffpunkt', '10:00-20:00'),
(7, 'Kaiserstraße', 'Fußgängerzone', '09:00-21:00'),
(8, 'Westliche', 'Stadtmitte', '08:00-22:00'),
(9, 'Wilhelmstraße', 'Altstadt', '09:00-17:00'),
(10, 'Bahnhofstraße', 'Bahnhofsnah', '06:00-23:00');
```

SQL-Implementierung – Tabelle Spot - Testfall

Testfall:

Query | Query History

```
1  SELECT SpotName FROM Spot LIMIT 3;
```

Ergebnis:

Data Output | Messages | Notifications

	spotname character varying (100)
1	Schlossplatz
2	Campus Süd
3	Marktplatz

SQL-Implementierung – Tabelle Vault

Tabelle erstellen:

Query | Query History

```sql
CREATE TABLE Vault ( -- Eine sichere, überwachte Übergabebox (z. B. ein Schließfach) oder künftig eigene Vaults mit Buchausgabe und Tracking-Funktion
    VaultID SERIAL PRIMARY KEY,
    StandortID INT NOT NULL REFERENCES Standort(StandortID),
    Name VARCHAR(50) NOT NULL, -- strategisch definierter Name,   z.B. "RED-1", "BLUE-2", etc.
    Sicherheitsstufe VARCHAR(50) NOT NULL, -- z.B. "Standard", "Hoch", "Premium"
    Status VARCHAR(50) NOT NULL
);
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Vault (StandortID, Name, Sicherheitsstufe, Status) VALUES
(1, 'RED-1', 'Hoch', 'aktiv'),
(2, 'BLUE-2', 'Mittel', 'aktiv'),
(3, 'GREEN-3', 'Niedrig', 'aktiv'),
(4, 'BLACK-4', 'Hoch', 'aktiv'),
(5, 'WHITE-5', 'Mittel', 'aktiv'),
(6, 'GOLD-6', 'Hoch', 'aktiv'),
(7, 'SILVER-7', 'Mittel', 'aktiv'),
(8, 'BRONZE-8', 'Niedrig', 'aktiv'),
(9, 'ONYX-9', 'Hoch', 'aktiv'),
(10, 'PEARL-10', 'Mittel', 'aktiv');
```

SQL-Implementierung – Tabelle Vault - Testfall

Testfall:

Query | Query History

```
1  SELECT Name, Sicherheitsstufe FROM Vault LIMIT 3;
```

Ergebnis:

Data Output | Messages | Notifications

	name character varying (50)	sicherheitsstufe character varying (50)
1	RED-1	Hoch
2	BLUE-2	Mittel
3	GREEN-3	Niedrig

SQL-Implementierung – Tabelle Zeitslot

Tabelle erstellen:

Query | Query History

```sql
CREATE TABLE Zeitslot ( -- Geplante Übergabezeiten = Vorgesehene Zeiträume für die Übergabe
    ZeitslotID SERIAL PRIMARY KEY,
    Startzeit TIME NOT NULL,
    Endzeit TIME NOT NULL
);
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Zeitslot (Startzeit, Endzeit) VALUES
('09:00','09:30'), ('09:30','10:00'),
('10:00','10:30'), ('10:30','11:00'),
('11:00','11:30'), ('11:30','12:00'),
('12:00','12:30'), ('12:30','13:00'),
('13:00','13:30'), ('13:30','14:00'),
('14:00','14:30'), ('14:30','15:00'),
('15:00','15:30'), ('15:30','16:00'),
('16:00','16:30'), ('16:30','17:00'),
('17:00','17:30'), ('17:30','18:00');
```

SQL-Implementierung – Tabelle Zeitslot - Testfall

Testfall:

Query | Query History

```sql
SELECT * FROM Zeitslot
WHERE startzeit BETWEEN '12:00:00' AND '16:00:00';
```

Ergebnis:

Data Output | Messages | Notifications

	zeitslotid [PK] integer	startzeit time without time zone	endzeit time without time zone
1	7	12:00:00	12:30:00
2	8	12:30:00	13:00:00
3	9	13:00:00	13:30:00
4	10	13:30:00	14:00:00
5	11	14:00:00	14:30:00
6	12	14:30:00	15:00:00
7	13	15:00:00	15:30:00
8	14	15:30:00	16:00:00
9	15	16:00:00	16:30:00

SQL-Implementierung – Tabelle BuchExemplar

Tabelle erstellen:

Query | Query History

```sql
-- Ein physisches Exemplar eines Buches, das vom Nutzer zur gemeinsamen Nutzung bereitgestellt wurde.
CREATE TABLE BuchExemplar (
    BuchExemplarID SERIAL PRIMARY KEY,
    BuchID INT NOT NULL REFERENCES Buch(BuchID),
    -- Ein Exemplar gehört genau zu einem Buch und einem Besitzer. Fremdschlüssel sorgen für Konsistenz und ermöglichen effiziente Abfragen.
    BesitzerID INT NOT NULL REFERENCES Benutzer(BenutzerID),
    Zustand VARCHAR(50) NOT NULL, -- z.B. „Sehr guter Zustand“, "guter Zustand", "gebrauchter Zustand", etc.
    Verfuegbarkeit VARCHAR(50) NOT NULL, -- z.B. "Verfügbar" oder "Nicht verfügbar"
    VaultID INT REFERENCES Vault(VaultID) -- Ein Exemplar kann optional einem Vault zugeordnet werden.
                                          -- Der Fremdschlüssel bildet diese Beziehung ohne Redundanz ab und unterstützt flexible Übergabeprozesse.
);
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO BuchExemplar (BuchID, BesitzerID, Zustand, Verfuegbarkeit, VaultID) VALUES
(1,1,'gut','verfügbar',1),
(2,2,'sehr gut','verfügbar',2),
(3,3,'gebraucht','verfügbar',3),
(4,4,'gut','verfügbar',4),
(5,5,'sehr gut','verfügbar',5),
(6,6,'gut','verfügbar',6),
(7,7,'gebraucht','verfügbar',7),
(8,8,'gut','verfügbar',8),
(9,9,'sehr gut','verfügbar',9),
(10,10,'gut','verfügbar',10),
(1,2,'gut','verfügbar',NULL),
(2,3,'gebraucht','verfügbar',NULL),
(3,4,'gut','verfügbar',1),
(4,5,'sehr gut','verfügbar',2),
(5,6,'gut','verfügbar',3),
(6,7,'gebraucht','verfügbar',4),
(7,8,'gut','verfügbar',5),
(8,9,'sehr gut','verfügbar',6),
(9,10,'gut','verfügbar',7),
(10,1,'gebraucht','verfügbar',8);
```

SQL-Implementierung – Tabelle BuchExemplar - Testfall

Testfall:

Query | Query History

```
1  SELECT * FROM BuchExemplar
2  WHERE vaultid IS NULL;
```

Ergebnis:

Data Output | Messages | Notifications

	buchexemplarid [PK] integer	buchid integer	besitzerid integer	zustand character varying (50)	verfuegbarkeit character varying (50)	vaultid integer
1	11	1	2	gut	verfügbar	[null]
2	12	2	3	gebraucht	verfügbar	[null]

SQL-Implementierung – Tabelle Ausleihe

Tabelle erstellen:

Query | Query History

```sql
-- Konkreter Ausleihvorgang eines BuchExemplars
CREATE TABLE Ausleihe (
    AusleiheID SERIAL PRIMARY KEY,
    BuchExemplarID INT NOT NULL REFERENCES BuchExemplar(BuchExemplarID),
    AusleiherID INT NOT NULL REFERENCES Benutzer(BenutzerID),
-- Die Ausleihe bildet den zentralen Prozess und verknüpft die beteiligten Entitäten miteinander.
-- Die Fremdschlüssel gewährleisten Datenkonsistenz, indem sie sicherstellen, dass
-- -    nur vorhandene Exemplare ausgeliehen werden können,
-- -    Ausleiher und Besitzer gültige Benutzer sind,
-- -    Übergaben ausschließlich an existierenden Spots oder Vaults erfolgen,
-- -    Zeitslots korrekt und widerspruchsfrei genutzt werden.
-- Damit bleibt die gesamte Prozesslogik konsistent und eindeutig abbildbar.
    Startdatum DATE NOT NULL,
    Enddatum DATE,
    SpotID INT REFERENCES Spot(SpotID), -- optional
    VaultID INT REFERENCES Vault(VaultID), -- optional
    ZeitslotID INT NOT NULL REFERENCES Zeitslot(ZeitslotID),
    Status VARCHAR(50) NOT NULL, -- z.B. "In Bearbeitung", "Geplant", "Abgeschlossen", etc.
    -- Der Übergabeort eines Ausleihvorgangs ist eindeutig festgelegt und kann entweder ein Spot oder ein Vault sein.
    CHECK (
        (SpotID IS NOT NULL AND VaultID IS NULL)
        OR
        (SpotID IS NULL AND VaultID IS NOT NULL)
    )
);
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Ausleihe (BuchExemplarID, AusleiherID, Startdatum, Enddatum, SpotID, VaultID, ZeitslotID, Status) VALUES
(1,2,'2024-03-01','2024-03-10',1,NULL,1,'abgeschlossen'),
(2,3,'2024-03-05',NULL,NULL,2,2,'aktiv'),
(3,4,'2024-03-07','2024-03-15',3,NULL,3,'abgeschlossen'),
(4,5,'2024-03-10',NULL,NULL,4,4,'aktiv'),
(5,6,'2024-03-12','2024-03-20',5,NULL,5,'abgeschlossen'),
(6,7,'2024-03-14',NULL,NULL,6,6,'aktiv'),
(7,8,'2024-03-15','2024-03-22',7,NULL,7,'abgeschlossen'),
(8,9,'2024-03-16',NULL,NULL,8,8,'aktiv'),
(9,10,'2024-03-17','2024-03-25',9,NULL,9,'abgeschlossen'),
(10,1,'2024-03-18',NULL,NULL,10,10,'aktiv'),
(11,3,'2024-03-19','2024-03-26',1,NULL,11,'abgeschlossen'),
(12,4,'2024-03-20',NULL,NULL,2,12,'aktiv'),
(13,5,'2024-03-21','2024-03-28',3,NULL,13,'abgeschlossen'),
(14,6,'2024-03-22',NULL,NULL,4,14,'aktiv'),
(15,7,'2024-03-23','2024-03-30',5,NULL,15,'abgeschlossen');
```

SQL-Implementierung – Tabelle Ausleihe - Testfall

Testfall:

Query | Query History

```
1  SELECT * FROM Ausleihe LIMIT 5;
```

Ergebnis:

Data Output | Messages | Notifications

	ausleiheid [PK] integer	buchexemplarid integer	ausleiherid integer	startdatum date	enddatum date	spotid integer	vaultid integer	zeitslotid integer	status character varying (50)
1	1	1	2	2024-03-01	2024-03-10	1	[null]	1	abgeschlossen
2	2	2	3	2024-03-05	[null]	[null]	2	2	aktiv
3	3	3	4	2024-03-07	2024-03-15	3	[null]	3	abgeschlossen
4	4	4	5	2024-03-10	[null]	[null]	4	4	aktiv
5	5	5	6	2024-03-12	2024-03-20	5	[null]	5	abgeschlossen

SQL-Implementierung – Tabelle Bewertung

Tabelle erstellen:

Query | Query History

```sql
CREATE TABLE Bewertung (
    BewertungID SERIAL PRIMARY KEY,
    AusleiheID INT NOT NULL REFERENCES Ausleihe(AusleiheID),
    BuchExemplarID INT NOT NULL REFERENCES BuchExemplar(BuchExemplarID),
    BewerterID INT NOT NULL REFERENCES Benutzer(BenutzerID),
    BewerteterID INT NOT NULL REFERENCES Benutzer(BenutzerID),
    Sterne INT NOT NULL CHECK (Sterne BETWEEN 1 AND 5),
    Kommentar VARCHAR(500),
    Bewertungsdatum DATE NOT NULL
);
-- Eine Bewertung ist immer einem konkreten Exemplar zugeordnet und wird von einem Benutzer abgegeben.
-- Die Fremdschlüssel stellen sicher, dass nur gültige Kombinationen entstehen und verhindern damit fehlerhafte oder unzulässige Bewertungen.
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Bewertung (AusleiheID, BuchExemplarID, BewerterID, BewerteterID, Sterne, Kommentar, Bewertungsdatum) VALUES
(1,1,2,1,5,'Sehr gute Übergabe','2024-03-11'),
(3,3,4,3,4,'Alles ok','2024-03-16'),
(5,5,6,5,5,'Top Zustand','2024-03-21'),
(7,7,8,7,3,'Etwas verspätet','2024-03-23'),
(9,9,10,9,5,'Super Kommunikation','2024-03-26'),
(11,11,3,2,4,'Guter Ablauf','2024-03-27'),
(13,13,5,4,5,'Sehr freundlich','2024-03-29'),
(15,15,7,6,4,'Alles bestens','2024-03-31'),
(1,1,2,1,5,'Wieder alles super','2024-04-01'),
(3,3,4,3,4,'Gerne wieder','2024-04-02');
```

SQL-Implementierung – Tabelle Bewertung - Testfall

Testfall:

Query | Query History

```
1  SELECT Sterne, Kommentar FROM Bewertung
2  WHERE sterne = 5;
```

Ergebnis:

Data Output | Messages | Notifications

	sterne integer	kommentar character varying (500)
1	5	Sehr gute Übergabe
2	5	Top Zustand
3	5	Super Kommunikation
4	5	Sehr freundlich
5	5	Wieder alles super

SQL-Implementierung – Tabelle Meldung

Tabelle erstellen:

Query | Query History

```sql
-- Eine Meldung erfasst ein von einem Benutzer gemeldetes Problem zu einem bestimmten BuchExemplar sowie zu einem betroffenen Benutzer (z. B. dem Besitzer).
CREATE TABLE Meldung (
    MeldungID SERIAL PRIMARY KEY,
    MelderID INT NOT NULL REFERENCES Benutzer(BenutzerID), -- der meldende Benutzer
    BetroffenesExemplarID INT NOT NULL REFERENCES BuchExemplar(BuchExemplarID), -- das in der Meldung genannte BuchExemplar
--  Meldungen verknüpfen Benutzer mit konkreten Exemplaren. Die Fremdschlüssel stellen sicher, dass nur existierende Benutzer und reale Exemplare gemeldet werden können.
    BetroffenerBenutzerID INT NOT NULL REFERENCES Benutzer(BenutzerID), -- der Benutzer, der von dem Problem betroffen ist
    Meldedatum DATE NOT NULL,
    Typ VARCHAR(100) NOT NULL, -- z.B. "beschädigt", "verloren", etc.
    Beschreibung VARCHAR(500)
);
```

Tabelle befüllen:

Query | Query History

```sql
INSERT INTO Meldung (MelderID, BetroffenesExemplarID, BetroffenerBenutzerID, Meldedatum, Typ, Beschreibung) VALUES
(1,2,2,'2024-03-10','Beschädigung','Kratzer entdeckt'),
(2,3,3,'2024-03-12','Verspätung','Buch kam zu spät zurück'),
(3,4,4,'2024-03-14','Verlust','Exemplar wurde verloren'),
(4,5,5,'2024-03-16','Beschädigung','Seiten eingerissen'),
(5,6,6,'2024-03-18','Verschmutzung','Kaffeeflecken im Buch'),
(6,7,7,'2024-03-20','Sonstiges','Unvollständige Rückgabe'),
(7,8,8,'2024-03-22','Beschädigung','Einband beschädigt'),
(8,9,9,'2024-03-24','Verspätung','Rückgabe stark verspätet'),
(9,10,10,'2024-03-26','Verlust','Exemplar nicht auffindbar'),
(10,1,1,'2024-03-28','Sonstiges','Unklare Übergabesituation');
```

SQL-Implementierung – Tabelle Meldung - Testfall

Testfall:

Query | Query History

```
1  SELECT Typ, Beschreibung FROM Meldung
2  WHERE typ = 'Beschädigung';
```

Ergebnis:

Data Output | Messages | Notifications

	typ character varying (100)	beschreibung character varying (500)
1	Beschädigung	Kratzer entdeckt
2	Beschädigung	Seiten eingerissen
3	Beschädigung	Einband beschädigt

Komplexer Testfall

Query | Query History

```sql
SELECT
    u.BenutzerID,
    u.Vorname || ' ' || u.Nachname AS Benutzername,

    -- Anzahl der Ausleihen dieses Benutzers
    (SELECT COUNT(*)
     FROM Ausleihe a
     WHERE a.AusleiherID = u.BenutzerID) AS AnzahlAusleihen,

    -- Durchschnittliche Bewertung, die der Benutzer erhalten hat
    (SELECT ROUND(AVG(bew.Sterne), 2)
     FROM Bewertung bew
     JOIN Ausleihe a2 ON bew.AusleiheID = a2.AusleiheID
     WHERE a2.AusleiherID = u.BenutzerID) AS Durchschnittsbewertung,

    -- Meldungen gegen den Benutzer (über seine Exemplare)
    (SELECT COUNT(*)
     FROM Meldung m
     JOIN BuchExemplar be ON m.BetroffenesExemplarID = be.BuchExemplarID
     WHERE be.BesitzerID = u.BenutzerID) AS MeldungenGegenIhn,

    -- Meldungen, die der Benutzer selbst erstellt hat
    (SELECT COUNT(*)
     FROM Meldung m2
     WHERE m2.MelderID = u.BenutzerID) AS MeldungenVonIhm,

    -- Datum der letzten Ausleihe
    (SELECT MAX(Startdatum)
     FROM Ausleihe a3
     WHERE a3.AusleiherID = u.BenutzerID) AS LetzteAusleihe

FROM Benutzer u
ORDER BY u.BenutzerID;
```

Komplexer Testfall – Ergebnis

Data Output Messages Notifications

	benutzerid [PK] integer	benutzername text	anzahlausleihen bigint	durchschnittsbewertung numeric	meldungengegenihn bigint	meldungenvonihm bigint	letzteausleihe date
1	1	Anton Muster	1	[null]	1	1	2024-03-18
2	2	Berta Muster	1	5.00	1	1	2024-03-01
3	3	Charlie Muster	2	4.00	1	1	2024-03-19
4	4	Dora Muster	2	4.00	1	1	2024-03-20
5	5	Emil Muster	2	5.00	1	1	2024-03-21
6	6	Frieda Muster	2	5.00	1	1	2024-03-22
7	7	Gustav Muster	2	4.00	1	1	2024-03-23
8	8	Hanna Muster	1	3.00	1	1	2024-03-15
9	9	Ida Muster	1	[null]	1	1	2024-03-16
10	10	Julius Muster	1	5.00	1	1	2024-03-17

Zusammenfassung

In Phase 2 wurde das ER-Modell vollständig in ein relationales Datenbankschema überführt und in PostgreSQL implementiert. Alle Tabellen wurden gemäß den Normalformen modelliert und mit realistischen Testdaten befüllt. Für jede Entität wurde ein Testfall definiert und erfolgreich ausgeführt.

Die Implementierung berücksichtigt alle fachlichen Anforderungen aus Phase 1 und erweitert das Modell dort, wo es für die technische Umsetzung notwendig war. Durch die klare Trennung von Büchern, Exemplaren, Ausleihen, Bewertungen und Meldungen entsteht ein robustes, erweiterbares Datenbanksystem.

Der komplexe Testfall bestätigt, dass die Beziehungen korrekt funktionieren und das System realistische Auswertungen ermöglicht. Damit ist die Grundlage für eine spätere produktive Anwendung vollständig geschaffen.

VIELEN DANK